

Module 5: Critical Thinking
Option 1: Improving the Accuracy of a Neural Network
Christine DeLuna
CSC 580: Dr. Nguyen
August 23, 2026

Introduction

In Module 4, a feedforward neural network was developed using the DeepChem Tox21 dataset to classify molecular compounds based on potential toxicity. The dataset represented each molecule using 1,024 molecular fingerprint features, allowing deep learning methods to be applied to a biomedical classification problem. The resulting model achieved a validation accuracy of approximately 90.92%. However, implementation also revealed a practical challenge associated with the evolution of machine learning frameworks. The instructional code relied heavily on TensorFlow 1.x programming conventions, whereas the current development environment uses TensorFlow 2.21. TensorFlow 2 introduced significant changes to the framework, including eager execution as the default behavior and greater integration of Keras as its high-level model-building interface (TensorFlow, 2026). Consequently, portions of the original Tox21 implementation required modernization to operate within the current TensorFlow, DeepChem, and RDKit environment.

The lessons learned during Module 4 provide the foundation for the current assignment. Rather than simply constructing a functioning Tox21 classifier, the objective is to investigate whether model performance can be improved through systematic hyperparameter optimization. Hyperparameters influence both the architecture and learning behavior of a neural network and can include characteristics such as network depth, number of hidden units, learning rate, dropout, batch size, and training duration (Goodfellow et al., 2016). The code provided for this assignment again uses legacy TensorFlow concepts, including computational graphs, placeholders, sessions, and the original TensorFlow optimizer interface. Rather than reintroducing deprecated programming methods, the intended experimental design is preserved while the implementation is adapted to TensorFlow 2.21 and the current Keras API. These software modifications are documented separately from changes intended to improve model performance so that modernization is not incorrectly interpreted as model optimization.

The experiment first establishes a Random Forest classifier as a non-neural-network baseline for the Tox21 classification task. Random Forest models combine predictions from multiple decision trees and can provide strong classification performance while reducing some of the overfitting associated with individual decision trees (Breiman, 2001). Neural-network performance is then evaluated using different combinations of hidden-layer size, network depth, learning rate, dropout, training epochs, batch size, and class weighting. An additional consideration is variability resulting from model initialization and stochastic training. Because identical neural-network architectures can produce different results across training runs, evaluating configurations across multiple random seeds provides a more reliable indication of whether an observed improvement results from the selected hyperparameters rather than a favorable initialization (Goodfellow et al., 2016).

This progression from Module 4 is particularly relevant to the development of AuVentures Health. The initial Tox21 implementation demonstrated how a neural network can learn relationships within complex biomedical data, whereas the current assignment shifts the focus toward understanding how reliably and efficiently the model learns those relationships. In healthcare applications, a high overall accuracy score alone is insufficient to establish model quality. Generalization to unseen data, class imbalance, variability across training runs, and the

consequences of different prediction errors must also be considered. Hyperparameter optimization therefore represents more than an attempt to maximize a single performance metric; it provides a structured method for examining how modeling decisions influence performance and reliability. These principles provide a foundation for evaluating future artificial intelligence models designed to identify meaningful patterns within complex and longitudinal healthcare data.

Methodology and Baseline Model Performance

The Tox21 dataset was loaded using DeepChem and divided into predefined training, validation, and testing sets. Each molecular compound was represented by 1,024 molecular fingerprint features, while the first toxicity task provided the binary classification target used for this experiment. The dataset also included sample weights, which were retained when calculating weighted classification accuracy. Maintaining separate training, validation, and testing datasets was important because each served a different purpose during model development. The training data were used to fit the models, the validation data were used to compare hyperparameter configurations, and the test data were reserved for evaluating the final model's ability to generalize to previously unseen compounds.

Before tuning the neural network, a Random Forest classifier containing 50 estimators and balanced class weighting was trained to establish a traditional machine learning baseline. As shown in Figure 1, the Random Forest achieved a weighted training accuracy of 99.74%, validation accuracy of 72.32%, and test accuracy of 71.97%. Although the extremely high training accuracy suggests that the model fit the training observations very closely, the similar validation and test results are particularly important. Performance decreased substantially when the model encountered unseen data, but it remained nearly identical between the validation and test sets. This suggests that the Random Forest provided a reasonably consistent estimate of how it would perform beyond the training data.

Figure 1

Random Forest Baseline Performance on the Tox21 Dataset

```

TensorFlow Version: 2.21.0
DeepChem Version: 2.8.0

Loading Tox21 dataset...

Dataset Shapes
Training features: (6258, 1024)
Training labels: (6258,)
Validation features: (782, 1024)
Validation labels: (782,)
Testing features: (783, 1024)
Testing labels: (783,)

=====
Random Forest Baseline
=====

About to fit model on training set...

Random Forest Results
Weighted Train Classification Accuracy: 0.9974
Weighted Validation Classification Accuracy: 0.7232
Weighted Test Classification Accuracy: 0.7197

Random Forest baseline complete.

Process finished with exit code 0
> assignments > CTA5-NN-Accuracy > tox21_nn_accuracy.py

```

Note. The Random Forest classifier achieved a weighted classification accuracy of 99.74% on the training set, 72.32% on the validation set, and 71.97% on the test set. The substantial difference between training and validation/test performance suggests that the model fits the training data extremely well but does not generalize at the same level to unseen molecular data. The similarity between validation and test accuracy indicates relatively consistent performance across the two unseen datasets.

Baseline Neural Network Performance

Following the Random Forest evaluation, a baseline fully connected neural network was developed to establish the starting point for hyperparameter optimization. The initial configuration followed the parameters provided in the assignment and consisted of one hidden layer containing 50 neurons, a learning rate of 0.001, a dropout rate of 0.50, 45 training epochs, and a batch size of 100. Positive-class weighting was retained to account for the weighting provided within the Tox21 dataset. A random seed of 456 was used for the initial run to support reproducibility.

As shown in Figure 2, the baseline neural network achieved a weighted validation classification accuracy of **62.43%**. This result was notably lower than the **72.32% validation accuracy** achieved by the Random Forest classifier. However, the purpose of this initial neural network was not to determine the final model, but rather to establish a controlled starting point from which the effects of hyperparameter changes could be evaluated. The difference in performance also demonstrated why a neural network should not be assumed to outperform a traditional machine learning algorithm simply because it provides a more complex architecture.

Figure 2

Baseline Tox21 Neural Network Performance Before Hyperparameter Optimization

```
=====
Tox21 Neural Network Hyperparameter Evaluation
=====
n_hidden      = 50
n_layers      = 1
learning_rate = 0.001
dropout_prob  = 0.5
n_epochs      = 45
batch_size    = 100
weight_positives = True
random_seed   = 456
=====

Training neural network...

Valid Weighted Classification Accuracy: 0.6243
```

Note. The baseline neural network used 50 hidden units, one hidden layer, a learning rate of 0.001, a dropout rate of 0.50, 45 training epochs, a batch size of 100, positive-class weighting, and a random seed of 456. The model achieved a weighted validation classification accuracy of 62.43%. This result establishes the initial neural-network performance against which subsequent hyperparameter configurations can be evaluated. The baseline accuracy was also lower than the 72.32% weighted validation accuracy achieved by the Random Forest classifier, indicating an opportunity to improve neural-network performance through systematic hyperparameter tuning.

The baseline result also highlighted the importance of hyperparameter selection in neural network development. Unlike model parameters that are learned during training, hyperparameters such as network depth, number of hidden units, learning rate, dropout, batch size, and training duration must be selected by the developer. A poorly suited combination can limit a network's ability to learn useful patterns or generalize to unseen observations. Therefore, the baseline accuracy of 62.43% should not be interpreted as evidence that neural networks were inherently unsuitable for the Tox21 classification task. Instead, it established the performance of one specific configuration and provided the justification for systematically exploring alternative settings.

Another consideration was the sensitivity of neural networks to random initialization. A single training run can produce a result that is unusually strong or weak because the model begins with randomly initialized weights. For this reason, the subsequent optimization process evaluated each hyperparameter configuration across multiple random seeds rather than selecting a model based on a single accuracy score. Averaging performance across repeated trials provided a more reliable basis for determining whether a particular configuration represented an actual improvement.

Hyperparameter Optimization

After establishing the baseline neural network, a systematic hyperparameter search was conducted to determine whether changes to the network architecture and training configuration could improve validation performance. The initial search focused on four hyperparameters: the number of hidden units, number of hidden layers, learning rate, and dropout rate. Hidden-layer sizes of 50 and 100 neurons were evaluated across networks containing one or two hidden layers. Learning rates of 0.001 and 0.0005 were tested along with dropout rates of 0.30 and 0.50. During this stage, the number of training epochs was held constant at 45, the batch size at 100, and positive-class weighting remained enabled. Limiting the initial search in this manner allowed the effects of network architecture, optimization rate, and regularization to be examined without simultaneously varying every available parameter.

Combining these values produced 16 unique hyperparameter configurations. Each configuration was trained three times using random seeds of 123, 456, and 789, resulting in 48 neural network training runs. The validation accuracy from the three trials was then averaged for each configuration. This repeated-seed methodology was important because neural network training involves stochastic processes, including weight initialization and dropout. Consequently, an unusually high result from a single training run does not necessarily indicate that the underlying hyperparameter configuration is superior. Evaluating mean performance across several initializations provided a more reliable basis for model comparison (Goodfellow et al., 2016).

As shown in Figure 3, the strongest configuration used 50 hidden units, two hidden layers, a learning rate of 0.001, and a dropout rate of 0.50. With epochs and batch size held at 45 and 100, respectively, this configuration achieved an average weighted validation accuracy of 65.49% with a standard deviation of 1.26% across the three random seeds. Compared with the baseline neural network accuracy of 62.43%, the initial optimization increased validation performance by approximately 3.06 percentage points, or approximately 4.9% relative to the baseline accuracy.

Figure 3

Best Tox21 Neural Network Configuration Following Initial Hyperparameter Optimization

```
=====
Training neural network...

Valid Weighted Classification Accuracy: 0.6104

Configuration Summary
-----
Seed Scores: ['0.6265', '0.5935', '0.6104']
Average Validation Accuracy: 0.6101
Standard Deviation: 0.0135

=====
BEST HYPERPARAMETER CONFIGURATION
=====
Hidden Units: 50
Hidden Layers: 2
Learning Rate: 0.001
Dropout: 0.5
Epochs: 45
Batch Size: 100
Average Validation Accuracy: 0.6549
Standard Deviation: 0.0126
=====

Process finished with exit code 0
```

Note. Sixteen neural-network configurations were evaluated using combinations of hidden-layer size, network depth, learning rate, and dropout. Each configuration was trained using three different random seeds, and validation accuracy was averaged across the three runs to reduce the influence of random initialization. The strongest configuration contained 50 hidden units across two hidden layers, a learning rate of 0.001, and a dropout rate of 0.50. With 45 epochs and a batch size of 100 held constant, the configuration achieved an average weighted validation accuracy of 65.49% ($SD = 1.26\%$). This represents an improvement over the 62.43% baseline neural network, although performance remained below the Random Forest validation accuracy of 72.32%.

Hyperparameter Refinement

Following the initial optimization, a second, narrower hyperparameter search was conducted using the strongest architecture identified during the first stage. The network configuration was held constant at 50 hidden units, two hidden layers, a learning rate of 0.001, and a dropout rate of 0.50. The refinement stage then evaluated the remaining training hyperparameters specified in the assignment: number of epochs, batch size, and positive-class weighting. Training durations of 30, 45, and 60 epochs were evaluated using batch sizes of 50 and 100, with positive-class weighting either enabled or disabled.

These values produced 12 additional configurations. As with the initial optimization, each configuration was evaluated using random seeds of 123, 456, and 789, resulting in 36 additional neural network training runs. Validation accuracy was averaged across the three runs for each configuration, and the standard deviation was calculated to evaluate consistency across random

initializations. This refinement strategy allowed the second experiment to focus on how the training process affected the strongest architecture rather than repeating a broad search across all previously evaluated parameters.

As shown in Figure 4, the strongest refined configuration retained the original architecture while increasing training duration to 60 epochs, reducing the batch size from 100 to 50, and retaining positive-class weighting. This configuration achieved an average weighted validation accuracy of 66.13%, with a standard deviation of 0.64%. The result represents a modest improvement over the 65.49% average validation accuracy obtained during the initial optimization. More notably, the standard deviation decreased from 1.26% to 0.64%, indicating that the refined configuration produced more consistent performance across the three random seeds.

Figure 4

Refined Tox21 Neural Network Hyperparameter Configuration

```
=====
Training neural network...
|
Valid Weighted Classification Accuracy: 0.5943

Refinement Configuration Summary
-----
Seed Scores: ['0.5950', '0.6111', '0.5943']
Average Validation Accuracy: 0.6001
Standard Deviation: 0.0078

=====
BEST REFINED HYPERPARAMETER CONFIGURATION
=====
Hidden Units: 50
Hidden Layers: 2
Learning Rate: 0.001
Dropout: 0.5
Epochs: 60
Batch Size: 50
Weight Positives: True
Average Validation Accuracy: 0.6613
Standard Deviation: 0.0064
=====

Process finished with exit code 0
```

Note. The strongest architecture identified during the initial hyperparameter search was further evaluated by varying training epochs, batch size, and positive-class weighting. Each configuration was trained using three random seeds. The best refined model contained 50 hidden units across two hidden layers, a learning rate of 0.001, a dropout rate of 0.50, 60 training epochs, a batch size of 50, and positive-class weighting. The configuration achieved an average weighted validation accuracy of 66.13% ($SD = 0.64\%$). This improved upon both the baseline

neural network accuracy of 62.43% and the initial optimized model's average accuracy of 65.49%, while demonstrating greater consistency across random initializations.

The refinement results demonstrate that model optimization involves more than identifying the configuration with the single highest observed accuracy. Consistency across repeated training runs is also an important consideration. A configuration that occasionally produces a high accuracy but varies substantially between random initializations may be less reliable than one that produces slightly lower but more stable performance. In this experiment, the refined model provided both a small improvement in average validation accuracy and reduced variability, supporting its selection as the final neural network configuration.

The improvement associated with 60 epochs also illustrates that additional training can be beneficial when evaluated within the context of validation performance rather than assumed to be advantageous by default. Similarly, the smaller batch size of 50 resulted in the strongest configuration among those evaluated. Positive-class weighting was retained in the final model, which is particularly relevant when working with biomedical classification data because overall accuracy can obscure differences in performance between classes. Removing weighting solely to obtain a higher aggregate accuracy could potentially favor the more common class rather than improve identification of the clinically relevant outcome.

Despite two stages of optimization, however, the neural network's average validation accuracy of 66.13% remained below the Random Forest baseline of 72.32%. This result suggested that further increasing neural-network complexity was not necessarily the appropriate path toward better performance. The final optimized configuration was therefore evaluated on the previously unseen test set to determine whether the improvements observed during validation generalized beyond the data used for hyperparameter selection.

Final Model Evaluation and Generalization

After completing both stages of hyperparameter optimization, the strongest configuration was selected for final evaluation. The optimized neural network contained 50 neurons in each of two hidden layers, a learning rate of 0.001, a dropout rate of 0.50, 60 training epochs, a batch size of 50, and positive-class weighting. A fixed random seed of 456 was used for the final training run to support reproducibility. Importantly, the test dataset was not used to select the hyperparameters. It was reserved until model selection was complete, allowing test performance to provide a more independent assessment of how well the optimized network generalized to unseen molecular compounds.

As shown in Figure 5, the final neural network achieved a weighted training accuracy of 99.62%, validation accuracy of 63.68%, and test accuracy of 65.00%. The test result was reasonably close to the 66.13% average validation accuracy observed during the three-seed refinement experiment. This suggests that the validation experiments provided a reasonable estimate of expected performance on unseen data. The lower validation accuracy of 63.68% in the final run was also not unexpected because the refinement result represented an average across three random initializations, whereas the final model represented a single reproducible training run.

Figure 5

Final Optimized Neural Network Performance on the Tox21 Test Set

```
=====
FINAL MODEL TEST EVALUATION
=====

Training final optimized model...

Final Optimized Neural Network Results
-----
Weighted Training Accuracy:  0.9962
Weighted Validation Accuracy: 0.6368
Weighted Test Accuracy:     0.6500
-----

Comparison with Previous Results
Baseline Neural Network Validation Accuracy: 0.6243
Optimized NN Average Validation Accuracy:    0.6613
Random Forest Validation Accuracy:           0.7232
=====

Process finished with exit code 0
```

Note. The final neural network was trained using the strongest hyperparameter configuration identified through the two-stage optimization process: 50 hidden units across two hidden layers, a learning rate of 0.001, a dropout rate of 0.50, 60 epochs, a batch size of 50, and positive-class weighting. Using a fixed random seed of 456, the final model achieved weighted classification accuracies of 99.62% on the training set, 63.68% on the validation set, and 65.00% on the previously unseen test set. The substantial difference between training and validation/test performance indicates that overfitting remained despite hyperparameter optimization and dropout regularization. However, test performance was relatively close to the 66.13% average validation accuracy observed during the three-seed refinement experiment, suggesting that the optimization results provided a reasonable estimate of generalization performance.

The difference between training and unseen-data performance, however, revealed an important limitation of the final model. Despite dropout regularization and systematic hyperparameter optimization, training accuracy reached 99.62%, while test accuracy remained at 65.00%. This approximately 34.6-percentage-point difference indicates substantial overfitting. The neural network learned patterns within the training observations extremely well but was considerably less successful at applying those learned relationships to new molecular compounds. Consequently, optimization improved the model relative to its original configuration but did not eliminate its generalization problem.

This distinction is important when evaluating model quality. If training accuracy alone had been considered, the optimized neural network could appear exceptionally successful at nearly 100% accuracy. Evaluation using separate validation and test sets demonstrates why that conclusion would be misleading. A useful predictive model must perform well on observations that were not

used to fit its parameters, particularly when the intended application involves previously unseen data (Goodfellow et al., 2016).

The final results also allowed the optimized neural network to be compared directly with the Random Forest baseline. The Random Forest achieved 72.32% validation accuracy and 71.97% test accuracy, compared with 63.68% and 65.00%, respectively, for the final neural network run. Although both models demonstrated very high training accuracy, the Random Forest maintained stronger performance on unseen data. In this experiment, therefore, hyperparameter optimization successfully improved the neural network, but it did not make the neural network the strongest classifier for the Tox21 data.

This comparison illustrates an important distinction between model optimization and model selection. Hyperparameter tuning asks how well a particular type of model can be configured for a problem, whereas model selection asks whether that type of model is the most appropriate approach in the first place. The results demonstrate that successfully optimizing a neural network does not necessarily establish that a neural network is preferable to a less complex machine learning method.

Conclusion

The results of this experiment demonstrate that hyperparameter optimization can meaningfully improve neural network performance, but they also illustrate an equally important principle: greater model complexity does not inherently produce better predictive performance. The baseline neural network achieved a weighted validation accuracy of 62.43%. Through systematic evaluation of network depth, hidden units, learning rate, dropout, training duration, batch size, and class weighting, the average validation accuracy increased to 66.13%. However, the final optimized neural network achieved only 65.00% weighted accuracy on the unseen test set and exhibited substantial overfitting, with training accuracy reaching 99.62%.

Perhaps the most interesting result was that the comparatively traditional Random Forest classifier outperformed the optimized neural network. The Random Forest achieved 72.32% validation accuracy and 71.97% test accuracy, demonstrating stronger generalization despite requiring substantially less architectural tuning. This finding is particularly relevant amid the current emphasis on artificial intelligence and increasingly complex deep learning architectures. The objective of machine learning should not be to use the most sophisticated model available, but to identify the model that most reliably solves the problem. For structured datasets such as the molecular fingerprint features used in this experiment, traditional machine learning approaches can remain highly competitive and may outperform more computationally complex neural networks.

This lesson also has important implications for healthcare AI. As I continue exploring machine learning applications through AuVentures Health, I do not believe every clinical problem requires a neural network or generative AI solution. Depending on the structure of the data and the question being asked, Random Forests, regression models, temporal methods, or other established approaches may provide better performance, greater interpretability, or more reliable generalization. More advanced deep learning models should therefore be introduced because the

characteristics of a problem require them, rather than because they represent the newest form of AI. Ultimately, responsible model development requires comparing approaches empirically and selecting the simplest model that can reliably accomplish the intended task.

References

Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5–32.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Mayr, A., Klambauer, G., Unterthiner, T., & Hochreiter, S. (2016). DeepTox: Toxicity prediction using deep learning. *Frontiers in Environmental Science*, 3, Article 80. <https://www.frontiersin.org/journals/environmental-science/articles/10.3389/fenvs.2015.00080/full>

TensorFlow. (2026). *Migrate from TensorFlow 1.x to TensorFlow 2*. <https://www.tensorflow.org/guide/migrate>

